Türkan Begüm Ahmetoğlu
31321

Report of PA3

1. Overview

This report describes the design and implementation of the Tour class, a thread-safe C++ header that synchronises POSIX threads (pthreads) representing visitors and an optional guide at a tourist attraction. Each thread calls arrive(), start(), and leave() in that fixed order. The implementation uses POSIX semaphores for all blocking/waiting conditions and a single mutex semaphore only for protecting shared counters in critical sections. No busy-waiting loops appear in any method.

2. Shared State

The class maintains the following shared fields:

- int inside – Number of visitors currently inside the attraction site.
- int required – Number of visitors needed to start a tour (excluding a guide).
- int hasGuide – 1 if this tour configuration includes a guide, 0 otherwise.
- int totalSlots – Total capacity of the site per tour: required + hasGuide.
- bool tourStarted – True once a full group has assembled and the tour is underway.
- int leavingCount – Number of participants (visitors + guide) that have already left the current tour.
- pthread_t guideThread – Thread ID of the thread assigned as the guide.
- sem_t mutex – Binary semaphore (initial value 1) protecting all shared counters.
- sem_t capacity – Counting semaphore (initial value totalSlots) that limits how many threads can be inside simultaneously and blocks all arrivals while a tour is in progress.
- sem_t guideDone – Counting semaphore (initial value 0) used to signal visitors that the guide has finished speaking.

3. Synchronisation Mechanisms

3.1 Semaphores Used

All blocking behaviour is implemented with POSIX semaphores (sem_wait / sem_post), so no thread ever busy-waits in a loop. The three semaphores and their roles are:

- mutex (init 1) – Mutual-exclusion semaphore. Wraps every read-modify-write on inside, tourStarted, and leavingCount to make those operations atomic.
- capacity (init totalSlots) – Counting semaphore acting as both a capacity limit and a tour gate. Each arriving thread calls sem_wait(&capacity), consuming one slot. When all totalSlots slots are consumed the semaphore value reaches 0 and every subsequent arriving thread blocks automatically no additional gate semaphore is needed. When the last participant leaves they call sem_post(&capacity) exactly totalSlots times, atomically restoring full capacity and unblocking the next waiting group.
- guideDone (init 0) – Used only when hasGuide == 1. The guide calls sem_post required times (once per visitor) after announcing the tour is over. Each visitor calls sem_wait once, so all visitors are held until the guide posts.

3.2 Mutex Usage

The mutex semaphore guards two critical sections: (a) incrementing inside and checking whether the group is complete in arrive(), and (b) incrementing leavingCount and resetting state in leave(). Using a semaphore as a binary mutex is consistent with the POSIX semaphore API and avoids introducing a separate pthread_mutex_t.

4. Method Pseudo-Code and Explanation

4.1 Constructor Tour(int n, int g)

Validates that n > 0 and g ∈ {0, 1}, throwing std::invalid_argument otherwise. Computes totalSlots = n + g. Initialises semaphores: mutex to 1, capacity to totalSlots, guideDone to 0. Sets inside = 0, tourStarted = false, leavingCount = 0.

4.2 arrive()

```
arrive():
  print "Arrived at the location."

  sem_wait(capacity)      // Block if site is full OR a tour is in progress

  sem_wait(mutex)         // Enter critical section
  inside++

  if hasGuide == 0:       // No-guide case
    if inside == required:
      tourStarted = true
```

```
    print "Enough visitors, tour is starting."
  else:
    print "Only <inside> visitors, starting solo shots."

else:               // Guide case
  if inside == totalSlots:
    guideThread = pthread_self()   // This thread becomes the guide
    tourStarted = true
    print "Enough visitors, tour is starting."
  else:
    print "Only <inside> visitors, starting solo shots."

sem_post(mutex)         // Leave critical section
```

Key design decisions in arrive():

- The capacity semaphore is consumed (sem_wait) by every arriving thread. Because it is initialised to totalSlots, exactly totalSlots threads can be inside simultaneously. All further arrivals block at sem_wait(capacity) without any additional gate semaphore.
- Non-completing threads do not post back to capacity; they simply hold their slot until they call leave(). This is the key difference from the previous design: there is no manual re-posting of a gate semaphore inside arrive(), which eliminates the race that allowed more than totalSlots threads inside at once.
- In the guide variant, the totalSlots-th thread to call arrive() is assigned as the guide (its pthread_t is saved) and the tour is started.

4.3 start()

Not implemented in Tour.h – declared only. Provided by the test harness. Simulates time spent at the attraction (waiting at the entrance or participating in an organised tour).

4.4 leave()

```
leave():
  sem_wait(mutex)
  if NOT tourStarted:
    print "Camera ran out of memory, I am leaving."
    inside--
    sem_post(capacity)    // Restore one slot
    sem_post(mutex)
```

```
  return

// Tour is in progress
isGuide = hasGuide AND pthread_equal(self, guideThread)
sem_post(mutex)

if isGuide:
  print "Tour guide speaking, the tour is over."
  for i in 0..required-1:
    sem_post(guideDone)  // Wake each visitor

else:  // Visitor
  if hasGuide:
    sem_wait(guideDone)  // Wait until guide announces end
  print "I am a visitor and I am leaving."

// Second critical section – track who has left
sem_wait(mutex)
leavingCount++
total = required + hasGuide

if leavingCount == total:
  print "All visitors have left, new visitors can come."
  inside            = 0
  leavingCount = 0
  tourStarted  = false
  for i in 0..totalSlots-1:
    sem_post(capacity)   // Restore full capacity for the next tour

sem_post(mutex)
```

Key design decisions in leave():

- A thread that finds tourStarted == false restores exactly one capacity slot with sem_post(capacity), allowing the next queued thread to enter, preserving the invariant that available slots always equals the number of free positions in the site.

- The mutex is released before sem_wait(guideDone) to prevent deadlock: if visitors held the mutex while waiting for the guide, the guide could never acquire it to call sem_post.
- The guide posts required times on guideDone one for each non-guide visitor so all visitors are unblocked simultaneously (or in any order).
- The second critical section increments leavingCount and, when all required + hasGuide participants have passed through, resets state and restores all totalSlots capacity slots with a loop of sem_post(capacity). This guarantees the site is fully cleared before any new visitor can enter, since capacity only reaches a positive value after all totalSlots posts are complete.

5. Correctness Arguments

5.1 No Busy-Waiting

Every blocking condition waiting for the site to have a free slot or for a tour to end (capacity), waiting for the guide to speak (guideDone), waiting for exclusive access to counters (mutex) is implemented with sem_wait. The OS puts the calling thread to sleep and wakes it only when the semaphore is signalled. There are no polling loops in arrive() or leave().

5.2 Mutual Exclusion

All reads and writes to inside, tourStarted, guideThread, and leavingCount occur while holding the mutex semaphore, ensuring atomicity and preventing data races.

5.3 No Deadlock

There is only one resource ordering: a thread acquires the mutex only for short, bounded critical sections and never holds it while waiting on another semaphore. Specifically, in leave() the mutex is released before sem_wait(guideDone), and the guide never waits on guideDone, so there is no circular wait.

5.4 Tour Isolation

The capacity semaphore starts at totalSlots. Exactly totalSlots threads consume all slots when a group forms; no further thread can enter until the last leaver restores all slots simultaneously with totalSlots sem_post calls inside a single mutex-protected block. This guarantees at most one active tour group in the site at any time, and the next group only enters after the previous one has fully vacated.

5.5 Guide Ordering

When hasGuide == 1, every visitor thread calls sem_wait(guideDone) before printing its leaving message. The guide calls sem_post(guideDone) exactly required times only after printing its

own message. Therefore the guide's output always appears before any visitor's leaving output, satisfying the correctness requirement.

5.6 Last-Visitor Notification

The leavingCount counter is incremented under the mutex in leave(). When it equals required + hasGuide, the last thread prints the everybody_left message and posts totalSlots times to capacity. Because this check is inside the mutex, exactly one thread will satisfy the condition and post exactly totalSlots times.

5.7 No-Tour (Camera-Out) Case

If a thread calls leave() and tourStarted is false, it decrements inside, posts once to capacity to allow the next thread in, and returns. This preserves the invariant that capacity correctly tracks site availability even when tours never form.